

DSAA2011 Machine Learning

Final Review (L07–L12)

Dr. Weikai Yang

Data Science and Analytics Thrust
& Computational Media and Arts Thrust
Information Hub
Hong Kong University of Science and Technology (Guangzhou)

April 29, 2026

Final exam logistics

- Time: Saturday, May 23, 2:00 – 3:30 PM.
- Venue: Lecture Hall A.
- Coverage: L01–L12, with focus on L07–L12.
- No cheat sheets or unauthorized materials are allowed.
- Bring your ID card and a basic non-programmable calculator.



How to use this review

- Read the frames in order to refresh the main ideas of every lecture.
- For each algorithm, recall: the **key idea**, the **main steps**, and the **strengths/limitations**.
- Pay attention to comparison tables: they often map directly onto exam-style questions.
- Cross-check every formula with the corresponding lecture slide before the exam.



Outline

- 1 L07: Ensemble methods
- 2 L08–L09: Clustering
- 3 L10: Dimensionality reduction
- 4 L11: Probabilistic graphical models
- 5 L12: Reinforcement learning



Outline

- 1 L07: Ensemble methods
- 2 L08–L09: Clustering
- 3 L10: Dimensionality reduction
- 4 L11: Probabilistic graphical models
- 5 L12: Reinforcement learning



Ensemble learning

- Aggregate multiple base learners to obtain a stronger predictor.
- Base learners should be **accurate** and **diverse**.
- For binary classification with N **independent** base learners and per-learner error $\varepsilon < 0.5$, the majority-vote error decreases as

$$\sum_{k=0}^{\lfloor N/2 \rfloor} \binom{N}{k} (1 - \varepsilon)^k \varepsilon^{N-k} \leq \exp \left(-\frac{1}{2} N (1 - 2\varepsilon)^2 \right).$$



Bagging

- Steps:
 - ▶ Draw B bootstrap samples from the training set.
 - ▶ Train a base classifier $\hat{f}_1, \dots, \hat{f}_B$ on each bootstrap sample.
 - ▶ Aggregate the base classifiers into a final predictor (voting or averaging).
- Reduces variance, while bias stays roughly the same.
 - ▶ Bias: systematic deviation $\mathbb{E}[\hat{f}_i] - g = \mathbb{E}[\hat{f}] - g$.
 - ▶ Variance: random fluctuations $\mathbb{E}[(\hat{f}_i - \mathbb{E}[\hat{f}_i])^2] = \mathbb{E}[(\hat{f} - \mathbb{E}[\hat{f}])^2]$.
 - ▶ Expected squared error: $\mathbb{E}[(g - \hat{f})^2] = \underbrace{(g - \mathbb{E}\hat{f})^2}_{\text{bias}^2} + \underbrace{\mathbb{E}[(\hat{f} - \mathbb{E}\hat{f})^2]}_{\text{variance}}.$
- Use **low-bias** base learners (e.g., deep trees) for bagging.
- Use **less correlated** base learners to maximize variance reduction.
 - ▶ Random forest: at each split, pick from a random subset of features to decorrelate trees.



Gradient Boosting

- Core idea: build new models to correct mistakes of the current ensemble.
- Each new model approximates the **negative gradient** of the loss.
 - ▶ Under MSE loss, the negative gradient equals the **residual** $r_i = y_i - \hat{y}_i$.
- Procedure:
 1. Fit an initial model $T^{(0)}$ on the training data.
 2. Compute the gradient of the loss with respect to current predictions.
 3. Fit a simple model $T^{(1)}$ to that gradient.
 4. Update the ensemble: $F \leftarrow F + \lambda T^{(1)}$.
 5. Repeat steps 2–4 until the stopping condition is met.
- Boosting reduces **bias** (whereas bagging reduces variance).
- Can overfit; common knobs: number of trees B , learning rate λ , tree depth d (often $d = 1$, i.e. stumps).
- Powers libraries such as XGBoost and LightGBM.



Adaptive Boosting (AdaBoost)

- Core idea: **adaptively reweight** samples so that misclassified ones receive more weight.
- Procedure:
 1. Initialize equal weights $w_i^{(1)} = 1/N$.
 2. At round t , train a base learner $T^{(t)}$ on weighted data $\{(w_1^{(t)}, x_1, y_1), \dots, (w_N^{(t)}, x_N, y_N)\}$ and compute the weighted error rate $\varepsilon^{(t)}$.
 3. Compute learner weight $\lambda^{(t)} = \frac{1}{2} \ln \left(\frac{1 - \varepsilon^{(t)}}{\varepsilon^{(t)}} \right)$.
 4. Update sample weights: $w_i^{(t+1)} \propto w_i^{(t)} \exp(-\lambda^{(t)} y_i T^{(t)}(x_i))$.
- Final model: $F = \sum_t \lambda^{(t)} T^{(t)}$.



Bagging vs. Boosting

	Bagging	Boosting
Training	Parallel	Sequential
Sampling	Bootstrap (with replacement)	Full data (reweighted)
Base learner	Low-bias, high-variance (e.g., deep trees)	Weak (e.g., stumps)
Diversity from	Different bootstrap samples (+random features in RF)	Correcting previous errors
Bias-variance	Reduces variance	Reduces bias
Overfitting	Resistant	Can overfit
Noise sensitivity	Robust	Sensitive
Examples	Bagging, Random Forest	Gradient Boosting, AdaBoost



Comparison: four ensemble methods

- **Bagging**: parallel training, bootstrap sampling plus averaging; reduces variance; weaker when trees stay highly correlated.
- **Random forest**: parallel training, bagging plus random feature subsets; stronger diversity than bagging; can miss informative features when k is too small.
- **Gradient boosting**: sequential training, fits residuals or negative gradients; often very accurate after tuning learning rate, depth, and number of trees.
- **AdaBoost**: sequential training, reweights difficult samples; simple and intuitive for classification; sensitive to noise and outliers.



Summary: ensemble learning

- Ensemble learning combines multiple base learners into a stronger predictor.
- Bagging reduces variance by averaging models trained on bootstrap samples.
- Random forest further reduces correlation by randomizing the feature subset at each split.
- Boosting builds learners sequentially so that later learners correct earlier mistakes.
- In practice, bagging and random forest are robust defaults; boosting tends to give higher accuracy after careful tuning.



Outline

- 1 L07: Ensemble methods
- 2 L08–L09: Clustering
- 3 L10: Dimensionality reduction
- 4 L11: Probabilistic graphical models
- 5 L12: Reinforcement learning



Recap: what is clustering

- Given a dataset $\mathcal{D}$ of n points, partition it into K clusters.
- Partition $\Delta = \{C_1, C_2, \dots, C_K\}$ minimizes a cost $\mathcal{L}(\Delta)$.
- No ground-truth labels during training (unsupervised).
- Two families of evaluation metrics:
 - ▶ External: require ground-truth labels.
 - ▶ Internal: no labels needed.



Recap: external clustering metrics

- **Purity** $= \frac{1}{N} \sum_{i=1}^k \max_j |C_i \cap L_j| \in [0, 1]$: fraction of points matching the **majority ground-truth class** in each cluster.
- **Caveat**: Purity = 1 does **not** imply the clustering matches the ground truth; over-clustering (e.g., every point in its own singleton cluster) trivially yields Purity = 1.
- **Rand Index (RI)** $= \frac{TP+TN}{\binom{n}{2}} \in [0, 1]$: fraction of **point pairs** correctly grouped or separated, where for any pair (i, j) :
 - ▶ TP: same class and same cluster; TN: different class and different cluster.
 - ▶ FP: different class but same cluster; FN: same class but different cluster.
- **ARI** $= \frac{RI - \text{Expected RI}}{\max(RI) - \text{Expected RI}} \in [-1, 1]$: RI **adjusted for chance**.
- **FMI** $= \sqrt{\text{Precision} \cdot \text{Recall}} \in [0, 1]$: geometric mean of pair-wise precision $\frac{TP}{TP+FP}$ and recall $\frac{TP}{TP+FN}$.
- **NMI** $= \frac{2 \cdot I(C, L)}{H(C) + H(L)} \in [0, 1]$: shared information between clusters C and labels L , normalized by their entropies.
- **V-measure**: harmonic mean of **Homogeneity** $(1 - \frac{H(L|C)}{H(L)})$ and **Completeness** $(1 - \frac{H(C|L)}{H(C)})$.



Recap: internal clustering metrics

- **Silhouette** $s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1]$: how similar a point is to its **own cluster** (a) compared to the **nearest other cluster** (b); overall score = average of $s(i)$.
- **Calinski-Harabasz (CH)** $= \frac{\text{Between-Cluster Dispersion}/(k - 1)}{\text{Within-Cluster Dispersion}/(N - k)}$: ratio of **between-** to **within-**cluster dispersion; **higher** is better.
- **Davies-Bouldin (DB)** $= \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \frac{S_i + S_j}{d(c_i, c_j)}$: average worst-case ratio of within-cluster scatter to between-cluster separation; **lower** is better.
- **Dunn** $= \frac{\min_{1 \leq i < j \leq k} d(C_i, C_j)}{\max_{1 \leq l \leq k} \text{diam}(C_l)}$: smallest inter-cluster distance over largest cluster diameter; **higher** is better.

All internal metrics evaluate clustering quality without ground truth.



Recap: k -Means and variants

- **k -Means**: given $X = \{x_1, \dots, x_n\}$, initialize centers $\{\mu_1, \dots, \mu_K\}$, then iterate:
 - ▶ **Assign**: $r_i = \arg \min_{k \in \{1, \dots, K\}} \|x_i - \mu_k\|$.
 - ▶ **Update**: $\mu_k = \frac{\sum_{i:r_i=k} x_i}{\sum_{i:r_i=k} 1}$.
- Objective (WCSS): $\mathcal{L}(\Delta, M) = \sum_{k=1}^K \sum_{i \in C_k} \|x_i - \mu_k\|^2$.
- **Monotonic decrease**: each iteration is non-increasing in $\mathcal{L}$, because
 - ▶ the **assign** step picks, for each x_i , its nearest current center, so $\mathcal{L}(\Delta', M) \leq \mathcal{L}(\Delta, M)$;
 - ▶ the **update** step sets μ_k to the cluster mean, the unique minimizer of $\sum_{x_i \in C'_k} \|x_i - \mu\|^2$ (why?), so $\mathcal{L}(\Delta', M') \leq \mathcal{L}(\Delta', M)$.



Recap: k -Means and variants

- Choosing K :
 - ▶ Elbow method—pick K where the marginal decrease in WCSS flattens (balance fit and over-clustering).
 - ▶ Use clustering metrics!
- Sensitive to initial centers; better initializations and variants:
 - ▶ k -**Means++**: choose the first center uniformly at random; for each subsequent center, sample a data point with probability $\propto$ squared distance to its nearest already-chosen center; spreads out centers and gives an $O(\log K)$ -competitive solution in expectation.
 - ▶ k -**Medoids**: use actual data points as centers; preferred when there are outliers, when only a non-Euclidean distance is meaningful (e.g., Manhattan, Hamming, edit distance), or when the mean is ill-defined (e.g., categorical features).



Recap: kernel k -Means

- **Idea**: run k -Means in a feature space induced by a kernel $K(x, y) = \langle \phi(x), \phi(y) \rangle$, without ever computing $\phi(x)$ explicitly.
- Cluster mean in feature space: $\mu_c = \frac{1}{n_c} \sum_{j \in C_c} \phi(x_j)$, with $n_c = |C_c|$.
- Squared distance from a point to a cluster mean (the assignment criterion) expands to a **kernel-only** expression:

$$\|\phi(x_i) - \mu_c\|^2 = K_{ii} - \frac{2}{n_c} \sum_{j \in C_c} K_{ij} + \frac{1}{n_c^2} \sum_{j, k \in C_c} K_{jk}.$$

- Algorithm: alternate (i) assign each point to the cluster with the smallest $\|\phi(x_i) - \mu_c\|^2$ and (ii) **implicitly** update μ_c via the new C_c ; iterate until assignments stop changing.
- Captures non-linear cluster shapes that linear k -Means cannot separate.



Recap: density-based methods

- **DBSCAN** parameters: neighborhood radius ε and minimum points minPts.
- For a point x , $N_\varepsilon(x) = \{y : d(x, y) \leq \varepsilon\}$. Then:
 - ▶ **Core point**: $|N_\varepsilon(x)| \geq \text{minPts}$.
 - ▶ **Border point**: not core, but lies within ε of some core point.
 - ▶ **Noise point**: neither core nor border.
- Clusters are maximal sets of points connected by chains of **density-reachable** core points; border points get attached to a neighboring core's cluster.
- **Limitation**: a single global ε cannot handle clusters of **widely varying density**.
- **OPTICS** addresses this by replacing the fixed ε with **core distance** and **reachability distance** per point; the resulting **reachability plot** reveals clusters at multiple density levels without retuning ε .



Recap: hierarchical methods

- Hierarchical clustering builds a **dendrogram** via repeated merges (agglomerative) or splits (divisive).
- Linkage choice defines how the distance $d(A, B)$ between two clusters A and B is measured:

Linkage	Distance $d(A, B)$	Characteristics
Single	$\min_{x \in A, y \in B} \ x - y\ $	Elongated “chain” clusters; sensitive to noise.
Complete	$\max_{x \in A, y \in B} \ x - y\ $	Compact, spherical clusters; less noise-sensitive.
Average	$\frac{1}{ A B } \sum_{x \in A, y \in B} \ x - y\ $	Balances single and complete; balanced cluster sizes.
Ward	$\frac{2 n_A n_B}{n_A + n_B} \ \mu_A - \mu_B\ ^2$	Compact clusters; minimizes within-cluster variance.

- Ward’s distance equals the increase in total within-cluster sum of squares caused by the merge.



Recap: soft clustering with GMMs

- A Gaussian Mixture Model represents data as a weighted sum of K Gaussian components:

$$p(x_i | \theta) = \sum_{k=1}^K \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k), \quad \sum_{k=1}^K \pi_k = 1.$$

- The latent variable $z_i \in \{1, \dots, K\}$ indicates which component generated x_i .
- **E-step** (compute responsibilities, with θ held fixed):

$$r_{ik} = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}, \quad \forall i, k.$$

- **M-step** (update parameters using r_{ik}):

$$\mu_k = \frac{\sum_i r_{ik} x_i}{\sum_i r_{ik}}, \quad \Sigma_k = \frac{\sum_i r_{ik} (x_i - \mu_k)(x_i - \mu_k)^\top}{\sum_i r_{ik}}, \quad \pi_k = \frac{\sum_i r_{ik}}{n}.$$

- Iterate E and M until the log-likelihood $\mathcal{L}(\theta) = \sum_i \log \sum_k \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)$ converges; EM is monotonically non-decreasing in $\mathcal{L}$.



Recap: choosing a clustering method

Method family	Works well when	Main caution
Prototype-based	Clusters are compact and well separated	Needs K and is geometry-sensitive
Density-based	Clusters are irregular and noise is present	Parameter choice matters
Hierarchical	Multi-resolution structure is useful	Computation can be expensive
GMM + EM	Soft membership is needed	Initialization can matter a lot

- The right choice depends on cluster shape, noise level, overlap, and whether soft assignments are needed.



Outline

- 1 L07: Ensemble methods
- 2 L08–L09: Clustering
- 3 L10: Dimensionality reduction**
- 4 L11: Probabilistic graphical models
- 5 L12: Reinforcement learning



Recap of dimensionality reduction

- Principal Component Analysis (PCA)
- Multidimensional Scaling (MDS)
 - Classical MDS
 - Variants: Metric MDS & Non-metric MDS
 - Isomap
- Linear Discriminant Analysis (LDA)
- SNE, t-SNE, UMAP
 - t-SNE
 - Uniform Manifold Approximation and Projection (UMAP) (optional)



Principal Component Analysis (PCA)

- **Key idea:** find **principal components** (linear combinations of existing dimensions) that capture the **most variance** in the data, equivalently, that **minimize reconstruction error**.
- **Key steps:**
 - **Standardize** the data $X_k \leftarrow (X_k - \mu_k) / \sigma_k$.
 - Compute the **covariance matrix** $\Sigma = \frac{1}{n} X^\top X$.
 - **Eigenvalue decomposition:** $\Sigma = U \Lambda U^\top$.
 - Project onto the **top- k eigenvectors (principal components)**: $Y = XU_k$, where $U_k = [e_1, \dots, e_k] \in \mathbb{R}^{p \times k}$.

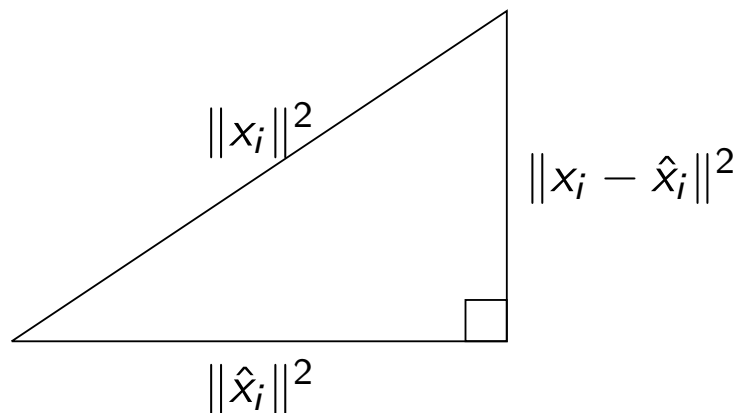


PCA: reconstruction error and projected variance

◉ Idea: minimizing reconstruction error $\Leftrightarrow$ maximizing projected variance.

- Data: $X \in \mathbb{R}^{n \times d}$ (n samples, d features).
- Projection matrix $U \in \mathbb{R}^{p \times k}$ has k orthonormal columns spanning the subspace.
- Projected points: $Z = XU$.
- Reconstructed points: $\hat{X} = ZU^\top = XU U^\top$.
- By Pythagoras, the reconstruction error satisfies

$$\underbrace{\sum_{i=1}^n \|x_i - \hat{x}_i\|^2}_{\text{reconstruction error}} = \underbrace{\sum_{i=1}^n \|x_i\|^2}_{\text{constant}} - \underbrace{\sum_{i=1}^n \|\hat{x}_i\|^2}_{\text{variance}}.$$



PCA: variance maximization

First consider projecting onto a one-dimensional subspace.

- Project onto a unit vector $u \in \mathbb{R}^d$: $z = Xu$, $z \in \mathbb{R}^n$.
- Variance of the projection:

$$\text{Var}(z) = u^\top \text{Cov}(X)u = u^\top \Sigma u, \quad \Sigma = \frac{1}{n}X^\top X.$$

- Optimization problem:

$$\max_u u^\top \Sigma u \quad \text{s.t.} \quad u^\top u = 1, \quad \text{equivalently} \quad \max_u \frac{u^\top \Sigma u}{u^\top u}.$$

Q: How is this solved? How does it generalize to a k -dimensional subspace ($k \geq 1$)?



Recap: linear algebra

Spectral decomposition

For an $n \times n$ real symmetric matrix A , $A = \sum_{i=1}^n \lambda_i e_i e_i^\top$, where λ_i is the i -th eigenvalue and e_i is the corresponding normalized eigenvector. Letting $\Lambda = \text{diag}\{\lambda_1, \dots, \lambda_n\}$ and $Q = [e_1, \dots, e_n]$, we have $A = Q\Lambda Q^\top$ and $A^{1/2} = Q\Lambda^{1/2}Q^\top$.

Maximization of quadratic forms on the unit sphere (Rayleigh quotient)

Let B be a positive semi-definite matrix with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0$ and normalized eigenvectors $e_1, \dots, e_n$, and let u be a unit vector. Then

- $\max_{\|u\|=1} u^\top B u = \lambda_1$, attained at $u = e_1$,
- $\min_{\|u\|=1} u^\top B u = \lambda_n$,
- $\max_{\|u\|=1, u \perp e_1, \dots, e_k} u^\top B u = \lambda_{k+1}$.

Proof sketch (top eigenvalue): write $u = \sum_j \alpha_j e_j$ with $\sum_j \alpha_j^2 = 1$. Then

 $u^\top B u = \sum_j \lambda_j \alpha_j^2 \leq \lambda_1 \sum_j \alpha_j^2 = \lambda_1$, with equality iff $\alpha_j = 0$ for all j such that $\lambda_j < \lambda_1$ (e.g., $u = e_1$).

PCA: principal components

Principal components

The eigenvectors of the covariance matrix define the principal components.

- **Covariance matrix:** $\Sigma = \frac{1}{n}X^\top X$ is symmetric.
- **Eigenvalue decomposition:** $\Sigma = U\Lambda U^\top$, with
 - ▶ $U \in \mathbb{R}^{p \times p}$: matrix of eigenvectors (principal components),
 - ▶ Λ : diagonal matrix of eigenvalues, $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$.
- **Top k components:** columns of $U_k \in \mathbb{R}^{p \times k}$ for the k largest eigenvalues.
- **Projection:** $Z = XU_k$, $Z \in \mathbb{R}^{n \times k}$.
- **Variance explained:** $\sum_{i=1}^k \lambda_i / \sum_{i=1}^p \lambda_i$.



PCA: things to keep in mind

Q1. Why do we (usually) need to do **standardization**?

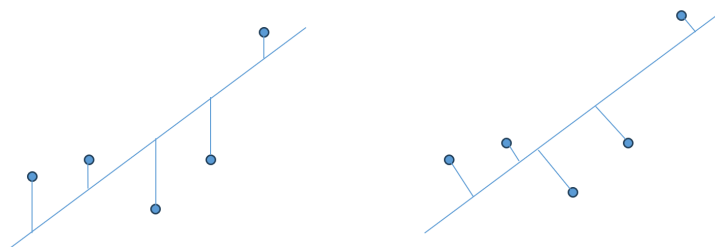
PCA is **not invariant** to rescaling individual features: rescaling changes Σ non-uniformly and can arbitrarily inflate/deflate a feature's contribution to a principal component, so unscaled PCA is sensitive to feature units.

Q2. Why use $\Sigma = \frac{1}{n}X^\top X$ instead of $\frac{1}{n-1}X^\top X$?

The factor only rescales eigenvalues by a constant; eigenvectors (the principal components) and the variance-explained ratio are identical.

Q3. How does **PCA** differ from **linear regression**?

PCA treats all dimensions symmetrically and minimizes the **perpendicular** reconstruction error onto the subspace. Linear regression treats y as special and minimizes the **vertical** squared residuals of y given X .



Kernel PCA

- Linear PCA covariance: $\Sigma = \frac{1}{n} \sum_{i=1}^n x_i x_i^\top = \frac{1}{n} X^\top X$.
- With the **kernel trick**: $\Sigma = \frac{1}{n} \sum_{i=1}^n \phi(x_i) \phi(x_i)^\top = \frac{1}{n} \phi(X)^\top \phi(X)$.

Q: How can we get its eigenvalues and eigenvectors using only the kernel $K_{ij} = \phi(x_i)^\top \phi(x_j)$?

- Each eigenvector satisfies $\frac{1}{n} \sum_{i=1}^n \phi(x_i) \phi(x_i)^\top v = \lambda v$.
 - Since $\phi(x_i)^\top v$ is a scalar, v is a linear combination of the $\phi(x_i)$:
there exists $\beta \in \mathbb{R}^{n \times 1}$ with $v = [\phi(x_1), \dots, \phi(x_n)] \beta = \phi(X) \beta$.
 - With $K = \phi(X)^\top \phi(X)$, the equation reduces to $K^2 \alpha = n \lambda K \alpha$, $\alpha \in \mathbb{R}^{n \times n}$.
- $\Rightarrow$ **Solution:** solve the eigenvalue problem $K \alpha = n \lambda \alpha$ (which also implies $K^2 \alpha = n \lambda K \alpha$).
- Normalize each PC by $\alpha_j \leftarrow \alpha_j / \sqrt{\lambda_j}$.
 - The projected dataset is $K \alpha[:, 1:k]$.



Classical Multidimensional Scaling (MDS)

- **Key idea:** preserve **pairwise** Euclidean distances between data points in the lower-dimensional space.
- **Key steps:**
 - Compute the **squared distance matrix** $D^{(2)}$ from the proximity matrix $D \in \mathbb{R}^{n \times n}$, where d_{ij} is the distance between samples i and j .
 - Compute the **inner-product matrix** via double centering: $B = -\frac{1}{2}JD^{(2)}J$, with $J = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$.
 - **Eigenvalue decomposition:** $B = U\Lambda U^\top$.
 - Coordinates: $X = U_k\Lambda_k^{1/2}$, where $U_k = [e_1, \dots, e_k] \in \mathbb{R}^{n \times k}$ and $\Lambda_k^{1/2} = \text{diag}(\sqrt{\lambda_1}, \dots, \sqrt{\lambda_k}) \in \mathbb{R}^{k \times k}$.



Strain function in MDS

- Quality of the MDS solution is evaluated by the **strain** function:

$$\text{Strain}(\mathbf{x}_1, \dots, \mathbf{x}_n) = \sqrt{\sum_{i < j} (b_{ij} - \mathbf{x}_i^\top \mathbf{x}_j)^2 / \sum_{i < j} b_{ij}^2}.$$

- Lower strain indicates a better fit.
- Classical MDS gives the **closed-form** minimizer of the strain.



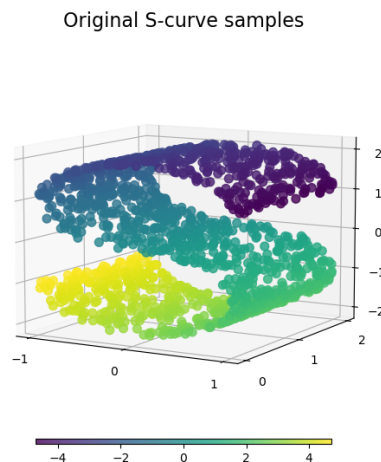
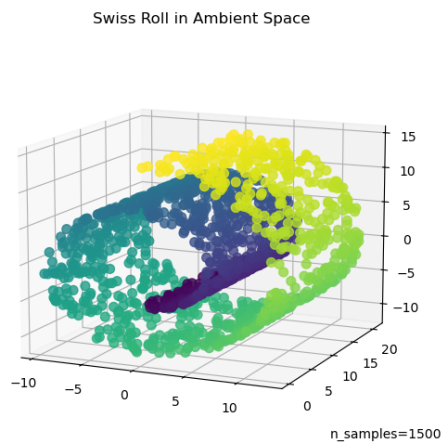
Metric MDS & Non-metric MDS

- **Metric MDS**: a generalization of classical MDS that directly minimizes a **stress function** to preserve distances:
 - $\text{Stress}(\mathbf{x}_1, \dots, \mathbf{x}_n) = \sqrt{\sum_{i < j} (d_{ij} - \|\mathbf{x}_i - \mathbf{x}_j\|)^2}$ or its normalized variant $\sqrt{\sum_{i < j} (d_{ij} - \|\mathbf{x}_i - \mathbf{x}_j\|)^2 / \sum_{i < j} \|\mathbf{x}_i - \mathbf{x}_j\|^2}$.
 - Optimized by **gradient descent or other iterative methods**.
- **Non-metric MDS**: preserves the **rank order** of dissimilarities rather than their exact values.
 - Uses **monotonic regression** to match dissimilarities in the low-dimensional space:
 $\text{Stress}(\mathbf{x}_1, \dots, \mathbf{x}_n, f) = \sqrt{\sum_{i < j} (f(d_{ij}) - \|\mathbf{x}_i - \mathbf{x}_j\|)^2}$ (or normalized).
 - Alternately optimize f and $\mathbf{x}_1, \dots, \mathbf{x}_n$.



Isomap

- Euclidean distances in MDS are not suitable when samples lie on a **curved manifold**.



- **Key idea:** use **shortest paths** on a nearest-neighbor graph to approximate **geodesic distances** on the manifold.
- **Key steps:**
 - Construct a k -NN (or ϵ -NN) graph.
 - Compute pairwise shortest-path distances on the graph (e.g., Dijkstra or Floyd-Warshall).



Linear Discriminant Analysis (LDA)

- **Key idea:** project data to maximize between-class scatter and minimize within-class scatter.
- **Steps:**
 - ▶ Compute within-class scatter $S_w = \sum_{j=1}^C \sum_{i \in C_j} (x_i - m_j)(x_i - m_j)^\top$ and between-class scatter $S_b = \sum_{j=1}^C n_j(m_j - m)(m_j - m)^\top$.
 - ▶ Solve the generalized eigenvalue problem $S_b v = \lambda S_w v$.
 - ▶ Project onto the top eigenvectors: $Y = XV \in \mathbb{R}^{n \times k}$, with $k \leq C - 1$.
- **Why at most $C - 1$ directions:** S_b is a sum of C rank-1 terms with the linear constraint $\sum_j n_j(m_j - m) = 0$, so $\text{rank}(S_b) \leq C - 1$ and the generalized problem has at most $C - 1$ non-zero eigenvalues.
- **Practical caveat:** when features outnumber samples, S_w may be singular and the eigenproblem becomes ill-posed (regularization or PCA-then-LDA is common).



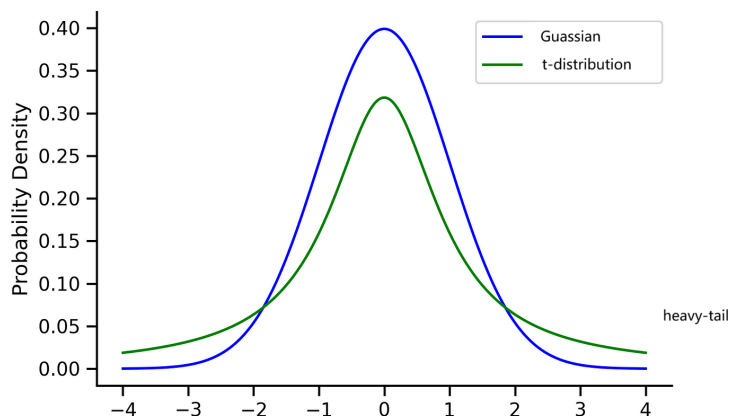
Stochastic Neighbor Embedding (SNE)

- **Key idea**: match **probability distributions** of pairwise similarities in high- and low-dimensional space.
- Build conditional similarities in both spaces from a Gaussian over distances; for each point i , $p_{j|i}$ is the chance of picking j as a neighbor.
- Train the embedding by minimizing the **KL divergence** between the high-dim distribution P and the low-dim distribution Q .
- KL is asymmetric, so SNE preserves **local** structure (small high-dim distances $\Rightarrow$ small low-dim distances).
- **Exam scope**: focus on the conceptual idea; specific similarity/loss formulas are not tested.



Crowding problem and t-distribution fix

- **Fact 1.** Eleven mutually equidistant points in a 10-D space cannot be modeled faithfully in a 2-D map.
 - **Fact 2.** Pairwise-distance distributions differ between high- and low-dimensional spaces because the volume of a ball grows as $V(r) \propto r^D$, leaving little room for “neighbors” in low dimensions.
- ⇒ Medium-distance points get pushed too far away (so $p_{ij} > q_{ij}$), creating many small attractive forces that pull point i toward the center of the map.
- **Fix:** increase q_{ij} for distant points by using a **t-distribution** (long-tail) in the low-dimensional space instead of the **Gaussian**.



t-Distributed Stochastic Neighbor Embedding (t-SNE)

- **Description**: address the crowding problem by replacing the low-dim Gaussian similarity with a **long-tail t-distribution**.
- Symmetrize the high-dim similarities to a joint distribution p_{ij} , and minimize the KL divergence between P and the t-distributed Q .
- Effect: preserves local structure tightly while letting moderately distant points spread out, yielding well-separated clusters.
- **Exam scope**: focus on the conceptual idea (Gaussian $\rightarrow$ t for low-dim, why this fixes crowding); formulas are not tested.



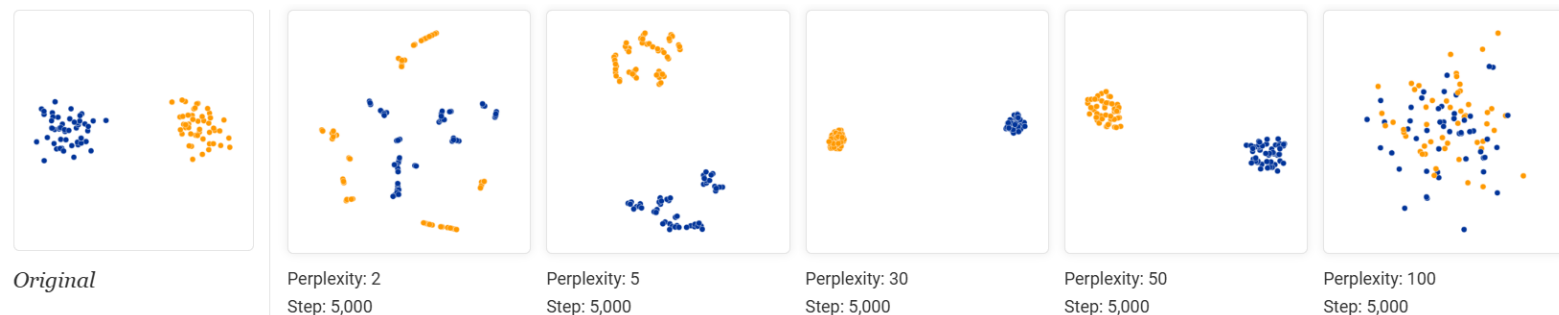
Uniform Manifold Approximation and Projection (UMAP)

- **Key idea:** build a **weighted nearest-neighbor graph** in the high-dim space and preserve its structure in the low-dim embedding.
- High-dim similarity comes from local neighbor distances; low-dim similarity uses a heavy-tailed kernel.
- Optimization minimizes a **cross-entropy loss** between the two graphs.
- Compared with t-SNE, UMAP is faster, scales better to large datasets, and tends to preserve more global structure.
- **Exam scope:** focus on the conceptual idea and the comparison with t-SNE; formulas are not tested.



How to use t-SNE effectively

- Reference: distill.pub/2016/misread-tsne/.
- Hyperparameters matter:



- **Perplexity**: balance between local and global structure.
 1. Effective number of neighbors used to compute conditional probabilities.
 2. **Low** perplexity (5–10): emphasizes local structure, gives tight clusters but may miss global relationships.
 3. **High** perplexity (30–50): captures more global structure, may merge nearby clusters.
- **Learning rate**: step size during gradient descent on the KL divergence.



Comparison of DR methods

Method	Type	Supervised	Input data	Best use case
PCA	Linear	Unsupervised	Feature	Preprocessing/compression
MDS	Linear/Nonlinear	Unsupervised	Distance	Preprocessing/compression
Isomap	Nonlinear	Unsupervised	Feature	Manifold visualization
LDA	Linear	Supervised	Feature	Classification
t-SNE, UMAP	Nonlinear	Unsupervised	Feature	Visualization

- Be ready to recall pros/cons, pseudocode, and the matrix worked on for each method.
- Note similarities (e.g., PCA $\equiv$ Classical MDS on Euclidean distances after centering, up to rotation/scale).



Outline

- 1 L07: Ensemble methods
- 2 L08–L09: Clustering
- 3 L10: Dimensionality reduction
- 4 L11: Probabilistic graphical models**
- 5 L12: Reinforcement learning



Recap of probabilistic graphical models (PGMs)

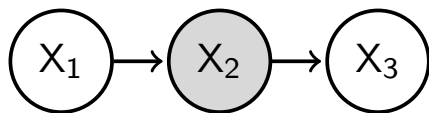
- Bayesian network
 - Chain, common cause (fork), common effect (collider, v-structure)
 - d-separation
 - Topic modeling (LDA)
 - Hidden Markov model (HMM)
- Markov random field (MRF)
 - Markov random field
- Factor graph



Bayesian network

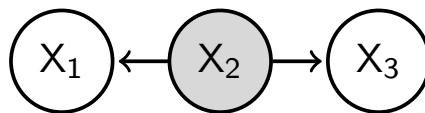
- Use a **directed acyclic graph (DAG)** to factor the joint distribution:

$$P(X_{1:d}) = \prod_{i=1}^d P(X_i \mid \text{pa}(X_i)).$$



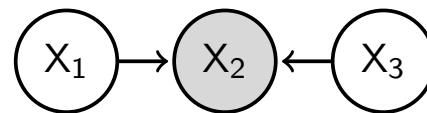
(a) Chain

$$X_1 \not\perp\!\!\!\perp X_3, X_1 \perp\!\!\!\perp X_3 \mid X_2$$



(b) Common cause (fork)

$$X_1 \not\perp\!\!\!\perp X_3, X_1 \perp\!\!\!\perp X_3 \mid X_2$$



(c) Common effect (v-structure)

$$X_1 \perp\!\!\!\perp X_3, X_1 \not\perp\!\!\!\perp X_3 \mid X_2$$

d-separation

- A, B, C are arbitrary non-intersecting sets of nodes; C may be empty.
- Question: does $A \perp\!\!\!\perp B \mid C$ hold?
- Construct all (undirected) paths from any node in A to any node in B .
- A path is **blocked** by C if either:
 - ▶ it contains a chain $i \rightarrow j \rightarrow k$ or a fork $i \leftarrow j \rightarrow k$ with the middle node $j \in C$, or
 - ▶ it contains a v-structure $i \rightarrow j \leftarrow k$ where neither j nor any descendant of j is in C .
- If **every path** from A to B is blocked, A is d-separated from B by C .
- **Theorem (d-separation)**: d-separation in the graph implies $A \perp\!\!\!\perp B \mid C$ in the joint distribution.

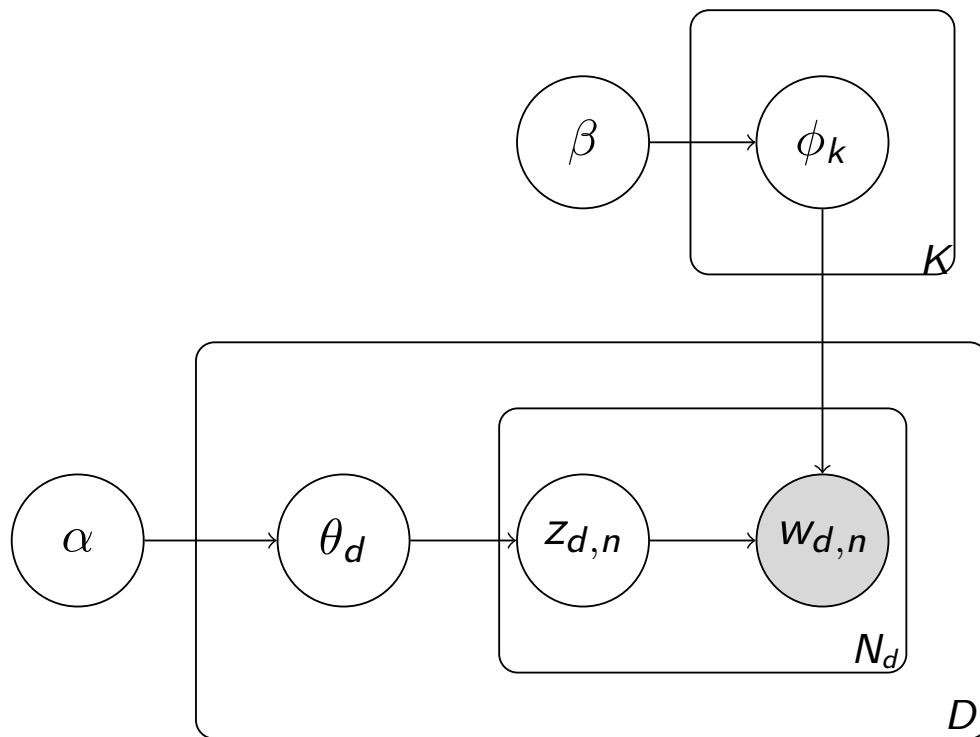


d-separation: rules of thumb for v-structures

- Chain $i \rightarrow j \rightarrow k$ and fork $i \leftarrow j \rightarrow k$: dependence is **active by default** and gets **blocked** when the middle node j is observed.
- **V-structure (collider)** $i \rightarrow j \leftarrow k$: dependence is **blocked by default** and becomes **active** when j or any descendant of j is observed.
 - ▶ This activation is the **explaining-away** phenomenon: two independent causes i and k become dependent once a common effect j is observed.
 - ▶ Conditioning on a **descendant** of the collider has the same effect as conditioning on the collider itself, because the descendant carries information about j .
- **Practical tip**: when checking $A \perp\!\!\!\perp B \mid C$, walk every path from A to B and apply the chain/fork/v-structure rule node-by-node; if at least one path remains active, the answer is “not independent.”



Latent Dirichlet Allocation (LDA)



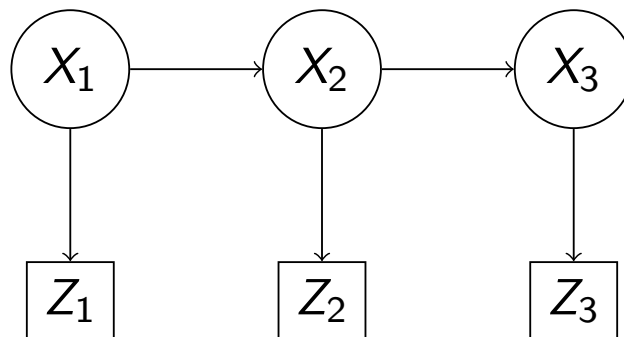
- β : Dirichlet prior over per-topic word distributions.
- ϕ_k : word distribution for topic k , $1 \leq k \leq K$.
- α : Dirichlet prior over per-document topic distributions.
- θ_d : topic distribution for document d , $1 \leq d \leq D$.
- $z_{d,n}$: topic of the n -th word in document d .
- $w_{d,n}$: the n -th word in document d (the only observed variable).



Dynamic Bayesian network: hidden Markov model (HMM)

- Variables at time t :
 - ▶ X_t : hidden state in $\mathcal{S} = \{s_1, \dots, s_n\}$ (latent),
 - ▶ Z_t : observation in $\mathcal{O} = \{o_1, \dots, o_m\}$ (observed).
- Transition probabilities $A = \{a_{ij}\}_{n \times n}$, $a_{ij} = P(x_{t+1} = s_j \mid x_t = s_i)$.
- Emission probabilities $B = \{b_{ij}\}_{n \times m}$, $b_{ij} = P(z_t = o_j \mid x_t = s_i)$.
- Initial state distribution $\pi = (\pi_1, \dots, \pi_n)$, $\pi_i = P(x_1 = s_i)$.
- Joint distribution

$$P(X_{1:T}, Z_{1:T}) = P(X_1) \prod_{t=2}^T P(X_t \mid X_{t-1}) \prod_{t=1}^T P(Z_t \mid X_t).$$



Markov random field (MRF)

- Use an **undirected graph** to model the joint distribution.
- Satisfies the **(local) Markov property**: a variable is conditionally independent of all others given its neighbors.
- Joint probability factorizes over maximal cliques:

$$P(X) = \frac{1}{Z} \prod_{C \in \text{max cliques}} \psi_C(X_C), \quad Z = \sum_x \prod_C \psi_C(X_C).$$

- Potential functions ψ_C encode interactions between variables.
 - ▶ Since ψ_C must be non-negative, a common parameterization is $\psi_C(X_C) = \exp(-H_Q(X_Q))$.
 - ▶ H_Q is real-valued, often chosen as

$$H_Q(x_Q) = \underbrace{\sum_{u \in Q} \alpha_u x_u}_{\text{linear term}} + \underbrace{\sum_{u, v \in Q, u \neq v} \beta_{uv} x_u x_v}_{\text{interaction term}}.$$



Factor graph

- A factor graph is a **bipartite** graph that represents the factorization of a function explicitly.
- **Variable nodes** (circles) connect to **factor nodes** (squares); edges indicate which variables enter each factor.
- It can describe both Bayesian networks and MRFs:
 - ▶ In a Bayesian network, each conditional/marginal becomes a factor.
 - ▶ In an MRF, each clique potential becomes a factor.
- Factor graphs are convenient for inference algorithms such as the **sum-product algorithm** and belief propagation.
- Some independence assumptions of directed graphs cannot be captured by undirected graphs (and vice versa); factor graphs help bridge the two.



Outline

- 1 L07: Ensemble methods
- 2 L08–L09: Clustering
- 3 L10: Dimensionality reduction
- 4 L11: Probabilistic graphical models
- 5 L12: Reinforcement learning**



Recap: what is reinforcement learning (RL)?

- RL is a **sequential decision-making** framework: an **agent** interacts with an **environment** and receives **rewards**.
- Goal: learn a policy that maximizes expected cumulative reward.
- Core trade-off: **exploration** (try uncertain actions) vs. **exploitation** (use the current best estimate).
- Difference from supervised learning: there are no labeled (x, y) pairs; the agent learns only from environment feedback.



Recap: multi-armed bandits (MAB)

- MAB is the simplest RL setting: a **single state** with L arms.
- At each round t , pull arm I_t and observe a random reward $W_t(I_t)$ from an unknown distribution.
- Three fundamental aims:
 - ▶ **Cumulative regret minimization**: $\text{regret} = \text{best-arm reward} - \text{realized reward}$.
 - ▶ **Best-arm identification / pure exploration**: fixed-confidence or fixed-budget.
 - ▶ **Simple regret minimization**: minimize the one-step regret of the recommended arm.
- Contextual bandits generalize MAB by letting the reward depend on observed context.



Recap: Markov decision process (MDP)

- An MDP is the standard formulation of RL with multiple states.
- It is defined by:
 - ▶ A set of states $s \in \mathcal{S}$ and actions $a \in \mathcal{A}$.
 - ▶ A **transition function** $T(s, a, s') = P(s' \mid s, a)$.
 - ▶ A **reward function** $R(s, a, s')$ (or $R(s, a)$, $R(s)$).
 - ▶ A start state s_0 and possibly a terminal state.
 - ▶ A **discount factor** $\gamma \in (0, 1]$ for infinite-horizon problems.
- **Markov property**: $P(S_{t+1} \mid S_t, A_t, S_{t-1}, A_{t-1}, \dots) = P(S_{t+1} \mid S_t, A_t)$.
- Discounted utility: $U([r_0, r_1, \dots]) = \sum_{t \geq 0} \gamma^t r_t \leq R_{\max}/(1 - \gamma)$.



Recap: policy and value functions

- A **policy** π specifies, for each state, which action to take (deterministic $\pi(s)$ or stochastic $\pi(a \mid s)$).
- State-value under policy π :

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t \geq 0} \gamma^t r_t \mid s_0 = s \right].$$

- Q-value under policy π :

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t \geq 0} \gamma^t r_t \mid s_0 = s, a_0 = a \right].$$

- An **optimal policy** π^* maximizes the expected return from every state.



Recap: Bellman equations

- Bellman expectation equations (fixed policy π):

$$V^\pi(s) = \sum_a \pi(a | s) Q^\pi(s, a),$$

$$V^\pi(s) = \sum_a \sum_{s'} \pi(a | s) T(s, a, s') [R(s, a, s') + \gamma V^\pi(s')].$$

- Bellman optimality equations:

$$V^*(s) = \max_a Q^*(s, a),$$

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')].$$

- Policy evaluation (iterative): $V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_k^\pi(s')]$, with cost $O(|\mathcal{S}|^2)$ per sweep.
- Policy evaluation (direct): without the max, the equations form a linear system in V^π .



Recap: finite vs. infinite horizon

- **Finite horizon**: terminate after T steps; the optimal policy is generally **nonstationary** (depends on time left).
- Solve by **backward induction**:

$$V_0(s) = 0, \quad V_n(s) = \max_a \left\{ R(s, a) + \sum_{s'} P(s' \mid s, a) V_{n-1}(s') \right\}.$$

- **Infinite horizon (discounted)**: the optimal policy is **stationary**, depending only on the current state.
- Alternative when no discount is desired: assume an absorbing terminal state reached with probability one.



Good luck in your final!

Good luck!

- This review is consolidated from the recap and summary frames of L07–L12 (26 Spring).

